Data Structures

Lecture 13: Linked List

Dr. Shabnam Samima

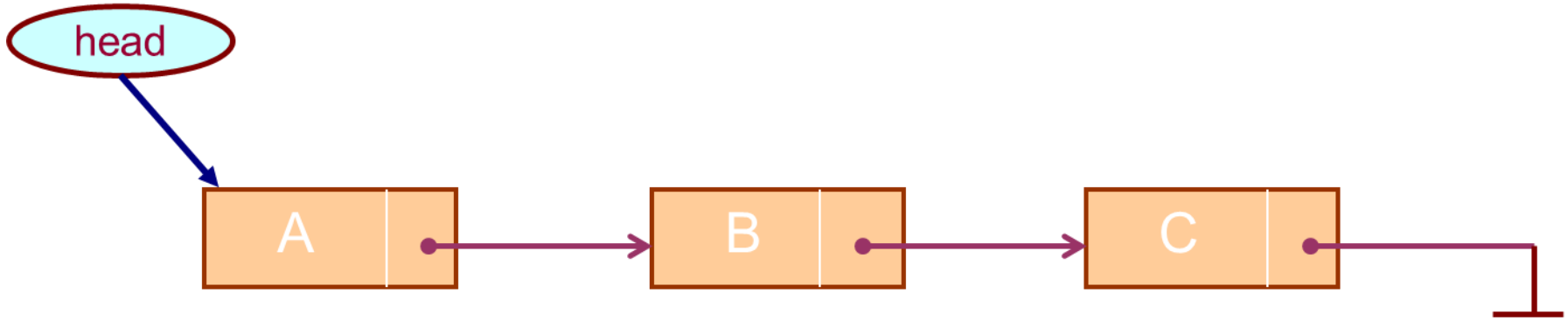
Content

- Introduction to Linked List

Introduction

A linked list, or one-way list, is a linear collection of data elements, called nodes, where the linear order is given by means of pointers.

- Successive elements are connected by pointers.
- Last element points to NULL.
- It can grow or shrink in size during execution of a program.
- It can be made just as long as required.
- It does not waste memory space.

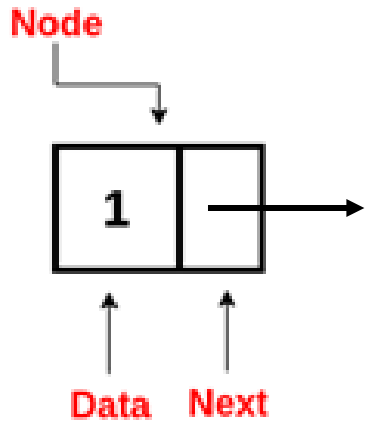


Linked List: Node

A node in a linked list typically consists of two components:

Data: It holds the actual value or data associated with the node. It may contain an entire record of data items (e.g., NAME, ADDRESS,…).

Next Pointer or Reference : It stores the memory address (reference) of the next node in the sequence.

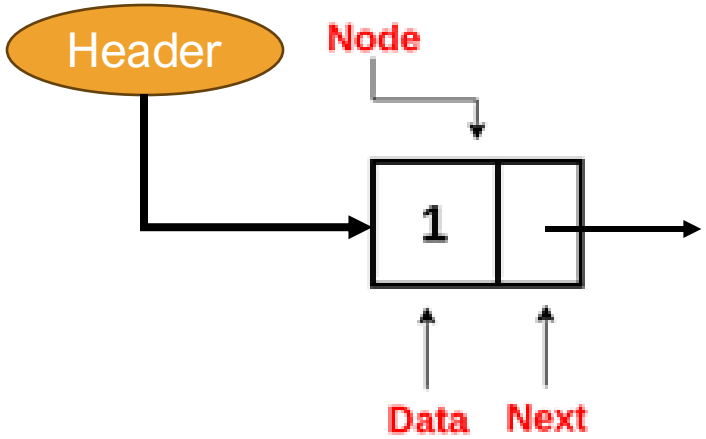



```
struct Node {
  // Data field
  int data;
  // Pointer to the next node
  Node* next;
}
```

The pointer of the last node contains a special value, called the null pointer, which is any invalid address.

Linked List: Node

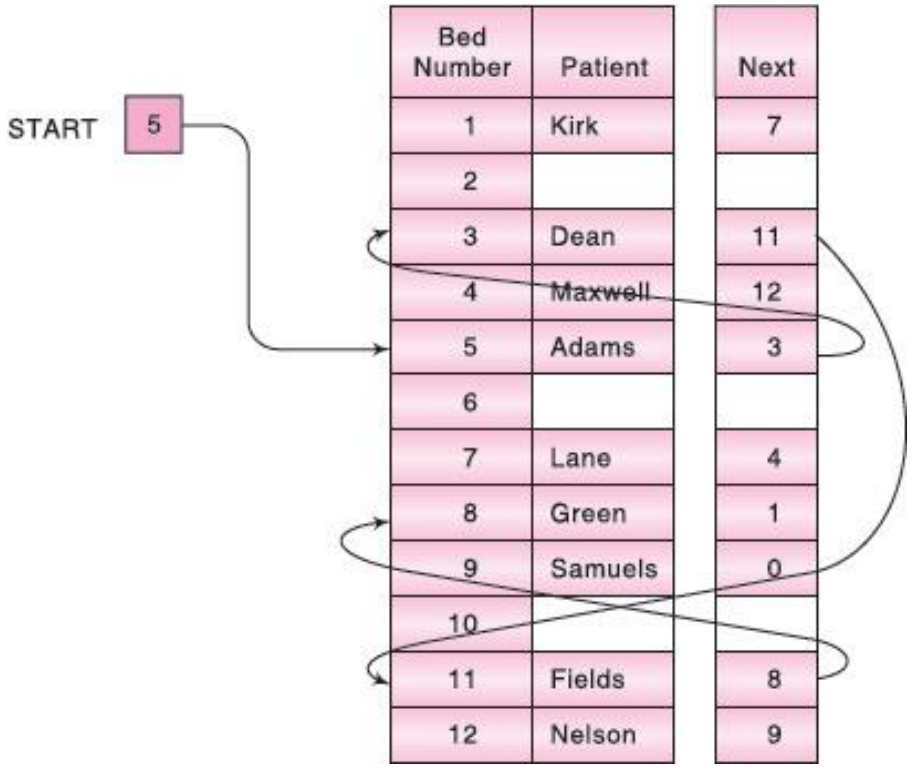
- Header/Start/Name: The linked list also contains a list pointer variable—called START/HEADER/NAME
- It contains the address of the first node in the list; hence there is an arrow drawn from START to the first node. Clearly, we need only this address in START to trace through the list.



A special case is the list that has no nodes. Such a list is called the null list or empty list and is denoted by the null pointer in the variable HEADER.

Linked List: Example

A hospital ward contains 12 beds, of which 9 are occupied. Suppose we want an alphabetical listing of the patients. This listing may be given by the pointer field, called Next. We use the variable START to point to the first patient.



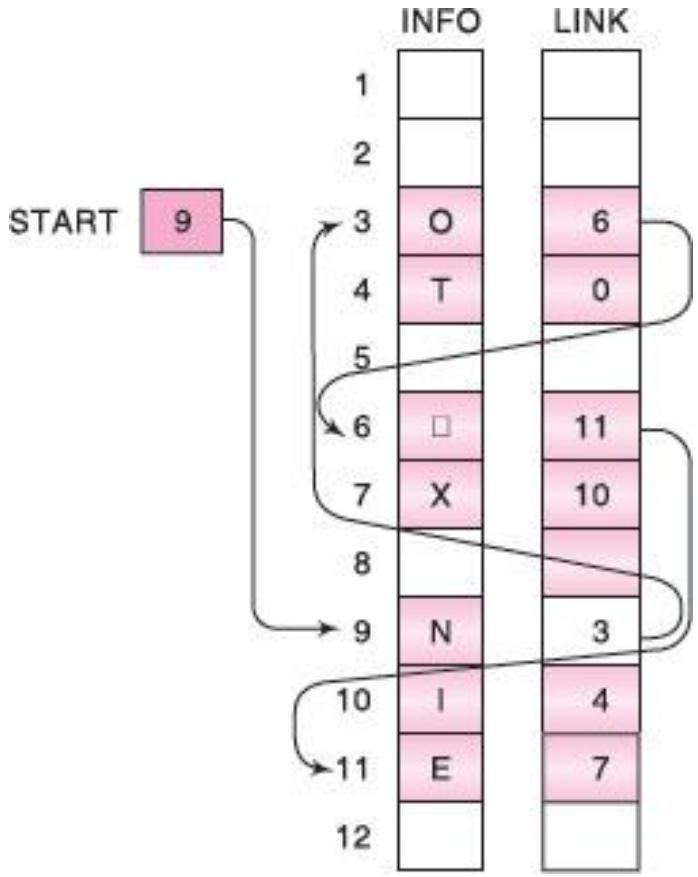
Linked List: Representation

There are two ways to represent a linked list in memory:	
Static representation using array	Dynamic representation using free pool of storage

Linked List: Static Representation

In static representation of a single linked list, two arrays are maintained: one array for data and the other for links

Example 1: a linked list in memory where each node of the list contains a single character. What is the string represented in this linked list?



NO EXIT

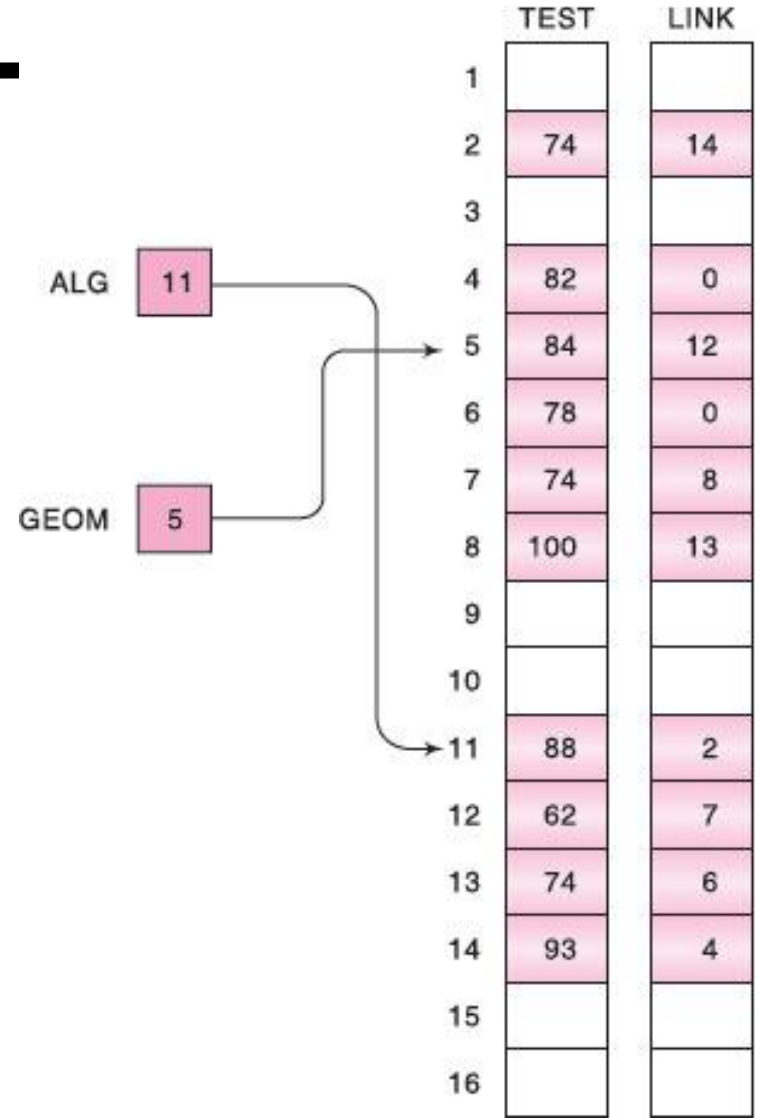
Linked List: Static Representation

This representation can be used to store multiple lists together as well

Example 2: The list shows test scores of ALG and GEOM, What are the test scores of ALG and GEOM in the same sequence as they are represented.

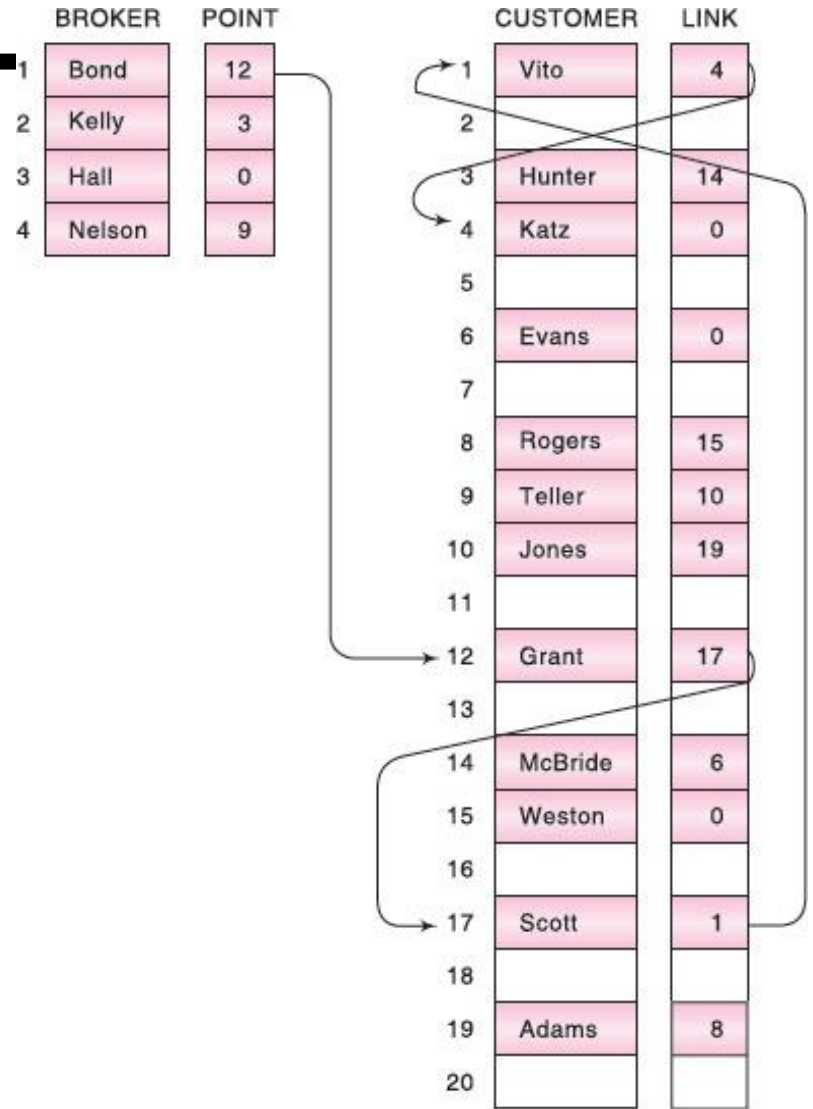
ALG: 88, 74, 93, 82

GEOM: 84, 62, 74, 100, 74, 78



Linked List: Static Representation

This representation can be used to store multiple lists together as well

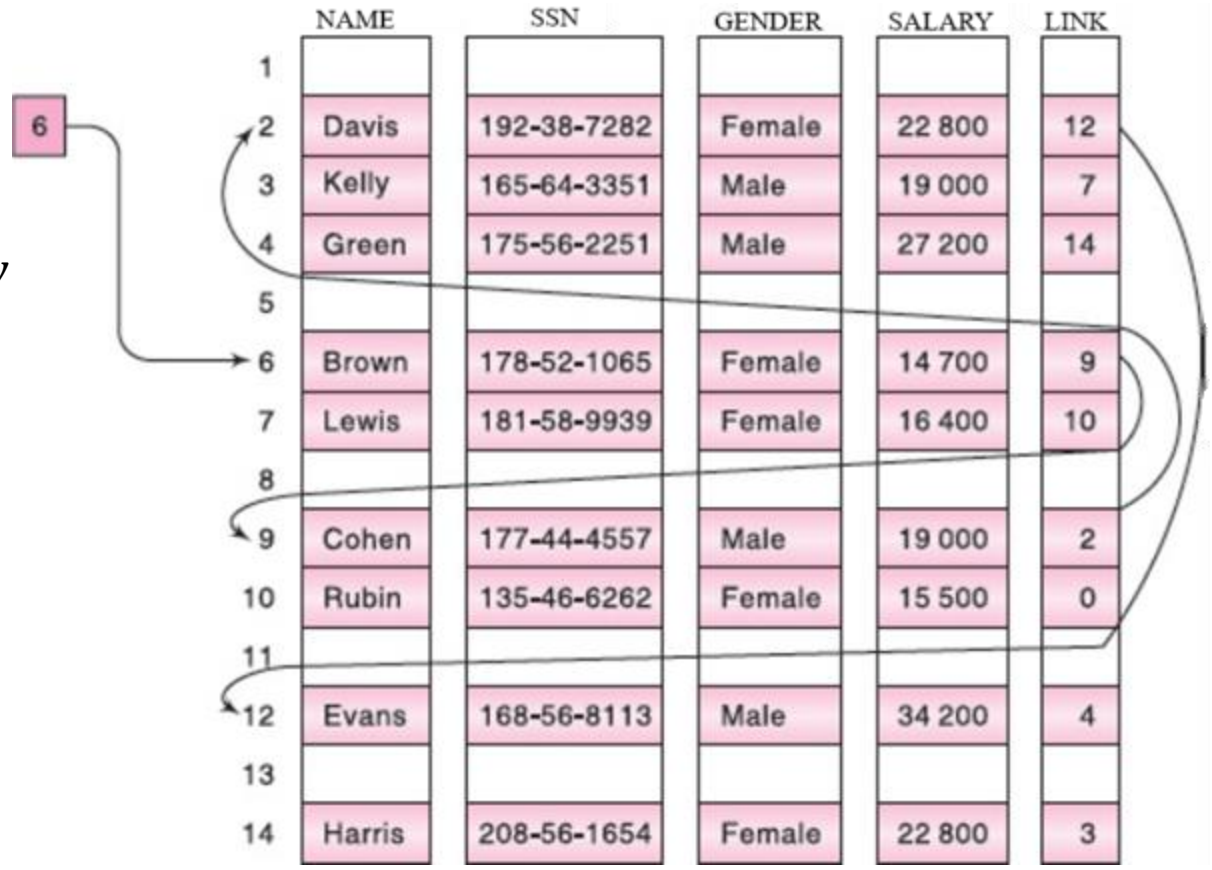


Example 3: Suppose a brokerage firm has four brokers and each broker has his own list of customers. How such a scenario can be represented using static representation

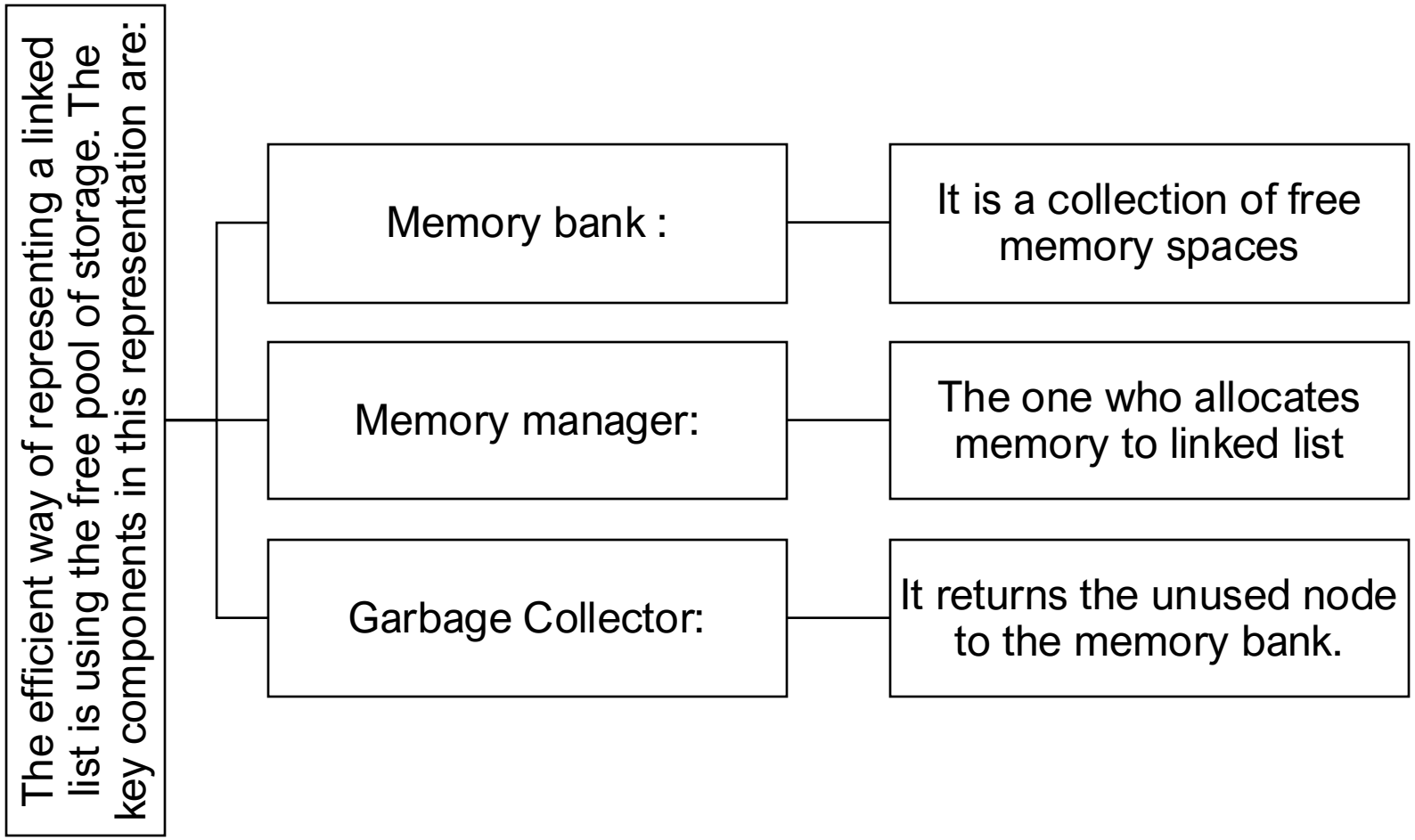
Question

Suppose the personnel file of a small company contains the following data on its nine employees:
Name, Social Security Number, Gender, Monthly Salary

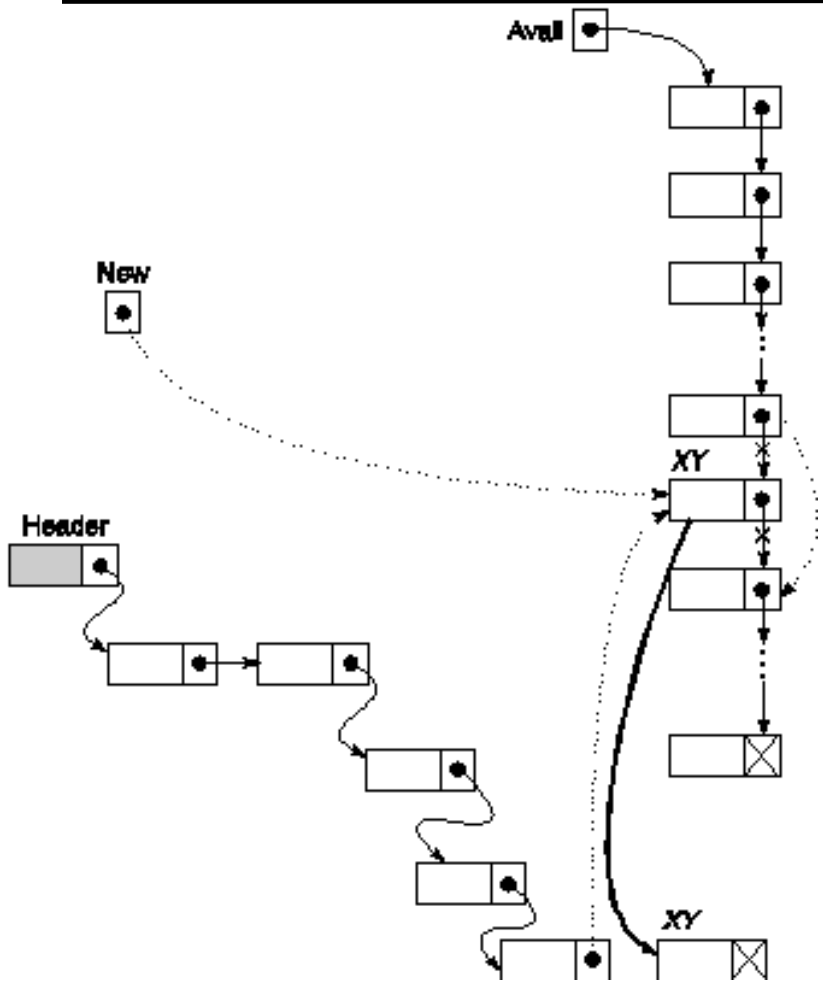
How can you represent the same using static representation of linked list?

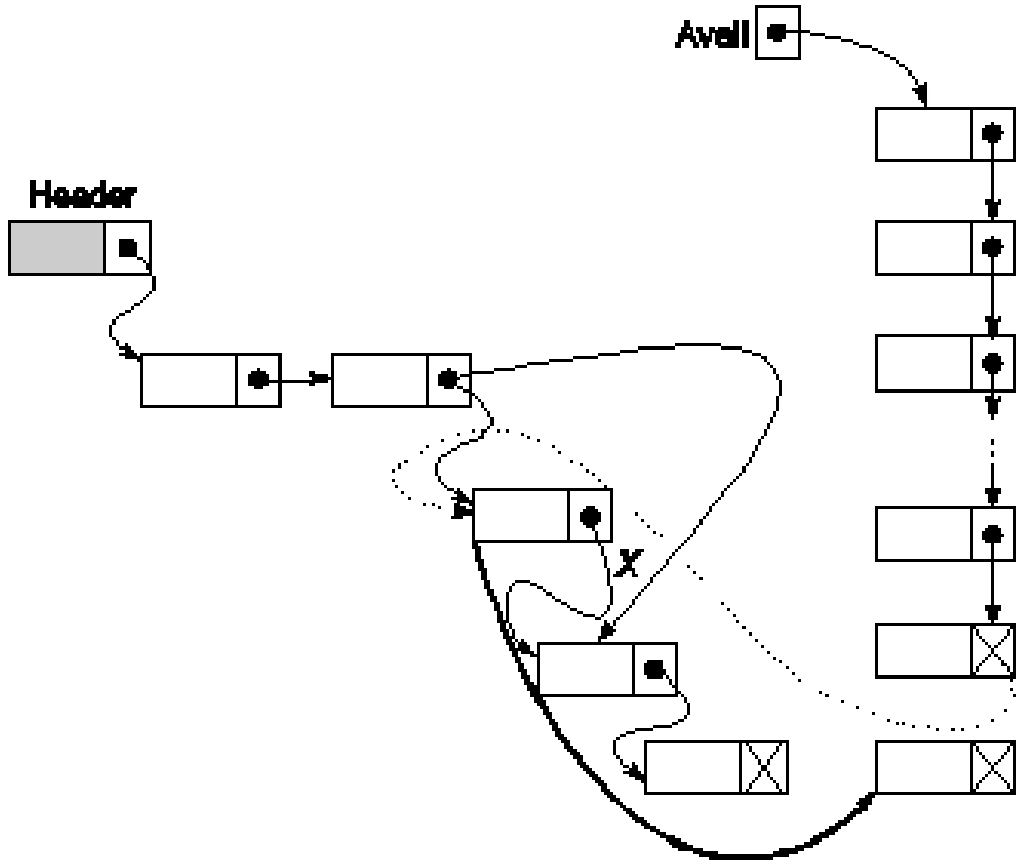


Linked List: Dynamic Representation



Linked List: Dynamic Representation





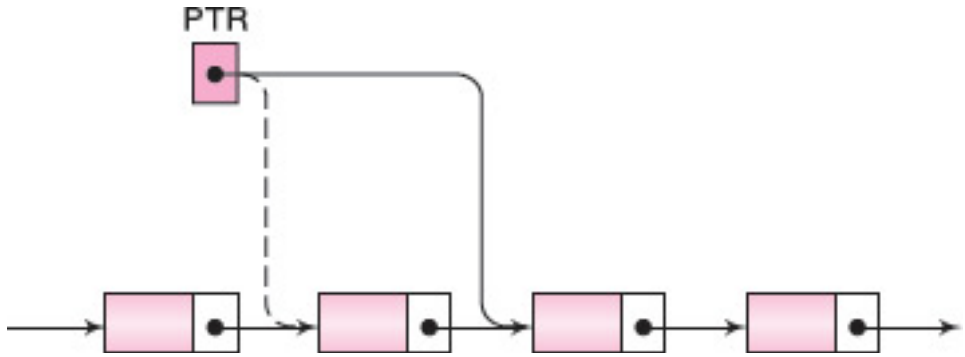
Linked List: Operations

- Traversing the list
- Searching for an element in the list
- Inserting a node into the list
- Deleting a node from the list
- Copying the list to make a duplicate of it
- Merging the linked list with another one to make a larger list

Linked List: Traversal

Let LIST be a linked list in memory stored with two components INFO and LINK with START pointing to the first element and NULL indicating the end of LIST.

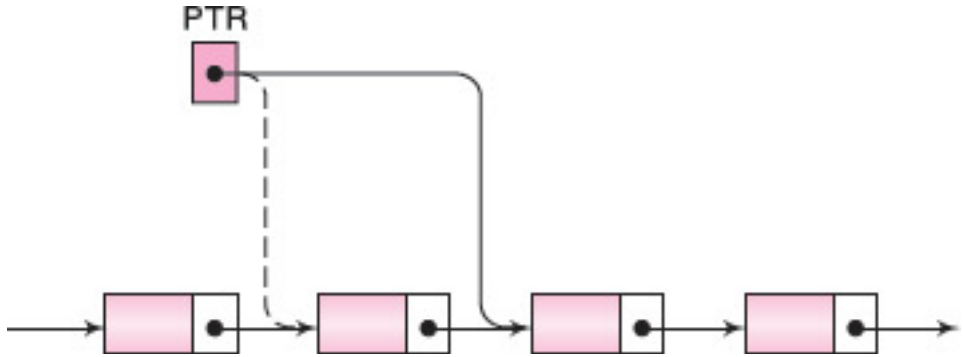
- A pointer variable PTR points to the node that is currently being processed.
- Accordingly, LINK[PTR] points to the next node to be processed.
- Thus the assignment PTR := LINK[PTR] moves the pointer to the next node in the list, as pictured in



Linked List: Traversal

Algorithm

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.



Question

Write the procedure/psuodcode which prints the information at each node of a linked list.

```
Procedure: PRINT(INFO, LINK, START)
This procedure prints the information at each node of the
list.
1. Set PTR := START.
2. Repeat Steps 3 and 4 while PTR ≠ NULL:
3. Write: INFO[PTR].
4. Set PTR := LINK[PTR]. [Updates pointer.]
5. [End of Step 2 loop.]
6. Return.
```